UPnP 미디어 서버 구현 분석

개요

rqbit의 UPnP 미디어 서버 기능은 `librqbit-upnp-serve` 크레이트에서 구현되며, DLNA 호환 미디어 서버를 제공하여 네트워크상의 미디어 플레이어(TV, 스마트폰, 태블릿 등)에서 토렌트 콘텐츠를 직접 스트리밍할 수 있게 합니다. 이는 UPnP AV 아키텍처와 DLNA 표준을 준수하는 완전한 미디어 서버 구현입니다.

UPnP AV 아키텍처 개요

1. 핵심 구성 요소

```
pub struct UpnpServer {
axum_router: Option<axum::Router>, // HTTP 서버 라우터
ssdp_runner: SsdpRunner, // SSDP 검색/알림 관리자
}
```

```
pub struct UpnpServerOptions {
pub friendly_name: String, // 사용자 친화적 이름
pub http_listen_port: u16, // HTTP 서버 포트
pub http_prefix: String, // URL 경로 접두사
pub browse_provider: Box<dyn ContentDirectoryBrowseProvider>, // 콘텐츠 제공자
pub cancellation_token: CancellationToken, // 종료 토큰
```

```
}
```

2. UPnP 디바이스 식별

```
fn create_usn(opts: &UpnpServerOptions) -> anyhow::Result<String> {
let mut buf = [0u8; 32];
let mut cursor = Cursor::new(&mut buf[..]);

// 호스트명, 서비스명, 포트, 경로를 시드로 사용
let _ = cursor.write_all(gethostname().as_encoded_bytes());
let _ = write!(&mut cursor, "{}{}{}",
opts.friendly_name, opts.http_listen_port, opts.http_prefix);

// 결정적 UUID 생성 (재시작 시에도 동일한 USN 보장)
let mut uuid = [0u8; 16];
rand::rngs::SmallRng::from_seed(buf).fill(&mut uuid);
let uuid = uuid::Builder::from_slice(&uuid)?.into_uuid();
Ok(format!("uuid:{}", uuid))
}
```

USN(Unique Service Name) 특징:

- **결정적 생성**: 동일한 설정으로 재시작 시 같은 UUID 생성
- **호스트 기반**: 호스트명을 포함하여 네트워크상 고유성 보장

- **설정 반영**: 서비스 설정 변경 시 새로운 UUID 생성

SSDP (Simple Service Discovery Protocol) 구현

1. 멀티캐스트 검색 및 알림

```
const SSDP_PORT: u16 = 1900;

const SSDP_MCAST_IPV4: SocketAddrV4 =

SocketAddrV4::new(Ipv4Addr::new(239, 255, 255, 250), SSDP_PORT);

const SSDP_MCAST_IPV6_SITE_LOCAL: SocketAddrV6 =

SocketAddrV6::new(Ipv6Addr::new(0xff05, 0, 0, 0, 0, 0, 0, 0xc), SSDP_PORT, 0, 0)


pub struct SsdpRunner {

opts: SsdpRunnerOptions,

socket: MulticastUdpSocket, // IPv4/IPv6 듀얼스택 소켓

}
```

2. M-SEARCH 요청 처리

```
#[derive(Debug)]

pub struct SsdpMSearchRequest<'a> {

pub host: &'a BStr,

pub man: &'a BStr, // "ssdp:discover"

pub st: &'a BStr, // 검색 대상 (upnp:rootdevice, MediaServer:1 등)
```

```
}

impl SsdpMSearchRequest<'_> {
fn matches_media_server(&self) -> bool {
if self.man != "\"ssdp:discover\"" {
return false;
}
// 루트 디바이스 또는 미디어 서버 검색에 응답
self.st == UPNP_DEVICE_ROOT || self.st == UPNP_DEVICE_MEDIASERVER
}
}
```

3. SSDP 응답 생성

```
fn generate_ssdp_discover_response(&self, st: &str, addr: SocketAddr) -> anyhow:
// 링크 로컬 주소는 VLC 호환성 문제로 거부
if matches!(addr.ip(), IpAddr::V6(a) if a.is_unicast_link_local()) {
debug!(?addr, "refusing to reply to a link-local address");
return Ok(None);
}
// 요청자와 같은 네트워크의 로컬 IP 결정
let local_ip = ::librqbit_upnp::get_local_ip_relative_to(addr, self.socket.nics(
let mut location = self.opts.description_http_location.clone();
```

```
let _ = location.set_ip_host(local_ip);


Ok(Some(format!(

"HTTP/1.1 200 OK\r

Cache-Control: max-age=75\r

Ext: \r

Location: {location}\r

Server: {server}\r

St: {st}\r

Usn: {usn}::{st}\r

Content-Length: 0\r\n\r\n"

)))

}
```

4. 주기적 ALIVE 알림

```
async fn task_send_alive_notifies_periodically(&self) {

let mut interval = tokio::time::interval(self.opts.notify_interval);

loop {

interval.tick().await;

self.try_send_notifies(NTS_ALIVE).await; // "ssdp:alive"

}
```

```
}


fn generate_notify_message(&self, device_kind: &str, nts: &str, opts: &Multicast

format!(

"NOTIFY * HTTP/1.1\r

Host: {host}\r

Cache-Control: max-age=75\r

Location: {location}\r

NT: {device_kind}\r

NTS: {nts}\r

Server: {server}\r

USN: {usn}::{device_kind}\r

\r

"

)

}
```

HTTP 서버 및 라우팅

1. Axum 기반 라우터 구성

```
pub fn make_router(

friendly_name: String,
```

```
http_prefix: String,
upnp_usn: String,
browse_provider: Box<dyn ContentDirectoryBrowseProvider>,
cancellation_token: CancellationToken,
) -> anyhow::Result<axum::Router> {
let app = axum::Router::new()
// 디바이스 설명 XML
.route("/description.xml", get(description_xml))
// 서비스 설명 문서 (SCPD)
.route("/scpd/ContentDirectory.xml", get(content_directory_scpd))
.route("/scpd/ConnectionManager.xml", get(connection_manager_scpd))
// SOAP 제어 엔드포인트
.route("/control/ContentDirectory", post(content_directory_control))
.route("/control/ConnectionManager", post(connection_manager_control))
// 이벤트 구독 엔드포인트
.route_service("/subscribe/ContentDirectory", content_dir_subscription)
.route_service("/subscribe/ConnectionManager", conn_mgr_subscription)
.with_state(state);
Ok(app)
}
```

2. 디바이스 설명 XML 생성

```xml
<?xml version="1.0"?>
<root xmlns="urn:schemas-upnp-org:device-1-0">
<specVersion>
<major>1</major>
<minor>0</minor>
</specVersion>
<device>
<deviceType>urn:schemas-upnp-org:device:MediaServer:1</deviceType>
<friendlyName>{friendly_name}</friendlyName>
<manufacturer>{manufacturer}</manufacturer>
<modelName>{model_name}</modelName>
<UDN>{unique_id}</UDN>

<serviceList>
<service>
<serviceType>urn:schemas-upnp-org:service:ContentDirectory:1</serviceType>
<serviceId>urn:upnp-org:serviceId:ContentDirectory</serviceId>
<SCPDURL>{http_prefix}/scpd/ContentDirectory.xml</SCPDURL>
<controlURL>{http_prefix}/control/ContentDirectory</controlURL>
<eventSubURL>{http_prefix}/subscribe/ContentDirectory</eventSubURL>
</service>
<service>
```

```
<serviceType>urn:schemas-upnp-org:service:ConnectionManager:1</serviceType>

<serviceId>urn:upnp-org:serviceId:ConnectionManager</serviceId>

<SCPDURL>{http_prefix}/scpd/ConnectionManager.xml</SCPDURL>

<controlURL>{http_prefix}/control/ConnectionManager</controlURL>

<eventSubURL>{http_prefix}/subscribe/ConnectionManager</eventSubURL>

</service>

</serviceList>

<presentationURL>/</presentationURL>

</device>

</root>
```

ContentDirectory 서비스 구현

1. 브라우징 인터페이스

```
pub trait ContentDirectoryBrowseProvider: Send + Sync {

fn browse_direct_children(&self, parent_id: usize, http_hostname: &str) -> Vec<I

fn browse_metadata(&self, object_id: usize, http_hostname: &str) -> Vec<ItemOrCo

}


#[derive(Debug, Clone, PartialEq, Eq)]

pub enum ItemOrContainer {

Container(Container), // 폴더/디렉토리
```

```
Item(Item), // 파일/미디어 아이템
}
```

2. 컨테이너 및 아이템 구조

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Container {
pub id: usize,
pub parent_id: Option<usize>, // None은 루트 컨테이너만
pub children_count: Option<usize>,
pub title: String,
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Item {
pub id: usize,
pub parent_id: usize,
pub title: String,
pub mime_type: Option<mime_guess::Mime>,
pub url: String, // 스트리밍 URL
pub size: u64,
}
```

3. SOAP Browse 요청 처리

```rust
#[derive(Deserialize, Debug)]
pub struct ContentDirectoryControlRequest {
#[serde(rename = "ObjectID")]
pub object_id: usize,
#[serde(rename = "BrowseFlag")]
pub browse_flag: BrowseFlag,
#[serde(rename = "StartingIndex", default)]
pub starting_index: usize,
#[serde(rename = "RequestedCount", default)]
pub requested_count: usize,
}

#[derive(Deserialize, PartialEq, Eq, Debug)]
pub enum BrowseFlag {
BrowseDirectChildren, // 하위 항목 목록
BrowseMetadata, // 특정 항목 메타데이터
}
```

4. DIDL-Lite XML 응답 생성

```rust
pub(crate) fn render(items: impl IntoIterator<Item = ItemOrContainer>) -> String
```

```
fn item(item: &Item) -> Option<String> {
let mime = item.mime_type.as_ref()?;
let upnp_class = match mime.type_().as_str() {
"video" => "object.item.videoItem",
_ => return None, // 비디오만 지원
};
Some(format!(
r#"<item id="{id}" parentID="{parent_id}" restricted="1">
<dc:title>{title}</dc:title>
<upnp:class>{upnp_class}</upnp:class>
<res protocolInfo="http-get:*:{mime_type}:*" size="{size}">{url}</res>
</item>"#,
id = item.id,
parent_id = item.parent_id,
title = item.title,
upnp_class = upnp_class,
mime_type = mime,
url = item.url,
size = item.size
))
}


// DIDL-Lite 네임스페이스로 래핑
```

```
let items_encoded = format!(
r#"<DIDL-Lite xmlns="urn:schemas-upnp-org:metadata-1-0/DIDL-Lite/"
xmlns:dc="http://purl.org/dc/elements/1.1/"
xmlns:upnp="urn:schemas-upnp-org:metadata-1-0/upnp/">
{items}
</DIDL-Lite>"#
);
// Samsung TV 호환성을 위해 CDATA 대신 XML 이스케이프 사용
let items_encoded = quick_xml::escape::escape(items_encoded);
format!(soap_response_template, items_encoded = items_encoded, ...)
}
```

ConnectionManager 서비스 구현

1. 프로토콜 정보 제공

```
pub const SOAP_ACTION_GET_PROTOCOL_INFO: &[u8] =
b"\"urn:schemas-upnp-org:service:ConnectionManager:1#GetProtocolInfo\"";

// HTTP GET을 통한 미디어 스트리밍 지원 선언
const PROTOCOL_INFO_RESPONSE: &str = r#"
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/">
<s:Body>
```

```
<u:GetProtocolInfoResponse xmlns:u="urn:schemas-upnp-org:service:ConnectionManag

<Source>http-get:*:video/*:*,http-get:*:audio/*:*</Source>

<Sink></Sink>

</u:GetProtocolInfoResponse>

</s:Body>

</s:Envelope>

"#;
```

2. 연결 관리 (기본 구현)

```
match action.as_ref() {

SOAP_ACTION_GET_PROTOCOL_INFO => {

// 지원하는 프로토콜 정보 반환

([(CONTENT_TYPE, CONTENT_TYPE_XML_UTF8)], PROTOCOL_INFO_RESPONSE).into_response(

},

SOAP_ACTION_GET_CURRENT_CONNECTION_IDS => {

// 현재 연결 ID 목록 (빈 목록)

([(CONTENT_TYPE, CONTENT_TYPE_XML_UTF8)], "0").into_response()

},

SOAP_ACTION_GET_CURRENT_CONNECTION_INFO => {

// 연결 정보 (기본값)

([(CONTENT_TYPE, CONTENT_TYPE_XML_UTF8)], connection_info_xml).into_response()

},
```

```
SOAP_ACTION_PREPARE_FOR_CONNECTION | SOAP_ACTION_CONNECTION_COMPLETE => {
// 고급 연결 관리는 미구현
StatusCode::NOT_IMPLEMENTED.into_response()
},
}
```

이벤트 구독 및 알림 시스템

1. 구독 요청 처리

```
pub(crate) async fn subscribe_http_handler(
State(state): State<UnpnServerState>,
request: axum::extract::Request,
) -> impl IntoResponse {
let req = match SubscribeRequest::parse(request) {
Ok(sub) => sub,
Err(err) => return err,
};

let resp = state.handle_content_directory_subscription_request(&req);
crate::subscriptions::subscription_into_response(&req, resp)
}
```

2. SystemUpdateID 알림

```
pub async fn notify_system_id_update(
url: &url::Url,
sid: &str,
seq: u64,
system_update_id: u64,
) -> anyhow::Result<()> {
let body = super::get_system_update_id::render_notify(system_update_id);


let resp = reqwest::Client::builder()
.build()?
.request(Method::from_bytes(b"NOTIFY")?, url.clone())
.header("Content-Type", r#"text/xml; charset="utf-8""#)
.header("NT", "upnp:event")
.header("NTS", "upnp:propchange")
.header("SID", sid)
.header("SEQ", seq.to_string())
.body(body)
.send()
.await?;


if !resp.status().is_success() {
```

```
anyhow::bail!("{:?}", resp.status())

}

Ok(())

}
```

DLNA 호환성 고려사항

1. 미디어 타입 지원

```
fn item(item: &Item) -> Option<String> {

let mime = item.mime_type.as_ref()?;

let upnp_class = match mime.type_().as_str() {

"video" => "object.item.videoItem",

// 오디오, 이미지 등은 현재 미지원

_ => return None,

};

}
```

현재 지원 범위:

- **비디오**: video/* MIME 타입만 지원
- **스트리밍**: HTTP GET 기반 스트리밍
- **메타데이터**: 기본적인 제목, 크기 정보만 제공

2. 클라이언트 호환성 최적화

```
// Samsung TV 호환성을 위한 XML 이스케이프
let items_encoded = quick_xml::escape::escape(items_encoded);


// VLC 호환성을 위한 링크 로컬 주소 거부
if matches!(addr.ip(), IpAddr::V6(a) if a.is_unicast_link_local()) {
debug!(?addr, "refusing to reply to a link-local address");
return Ok(None);
}
```

3. 네트워크 최적화

```
// IPv4/IPv6 듀얼스택 지원
let socket = MulticastUdpSocket::new(
(Ipv6Addr::UNSPECIFIED, SSDP_PORT).into(),
SSDP_MCAST_IPV4,
SSDP_MCAST_IPV6_SITE_LOCAL,
None,
None,
).await?;
```

성능 및 확장성

1. 비동기 처리

```
pub async fn run_forever(&self) -> anyhow::Result<()> {
let t0 = self.try_send_example_msearch();
let t1 = self.task_respond_on_msearches();
let t2 = self.task_send_alive_notifies_periodically();


let wait = async move {
tokio::join!(t0, t1, t2);
Ok(())
};


tokio::select! {
r = wait => r,
_ = self.opts.shutdown.cancelled() => {
self.try_send_notifies(NTS_BYEBYE).await; // 정상 종료 알림
Ok(())
}
}
}
```

2. 메모리 효율성

- **템플릿 기반**: 컴파일 타임에 포함된 XML 템플릿 사용
- **스트림 처리**: 대용량 미디어 파일을 위한 HTTP 범위 요청 지원

- **최소 상태**: 필요한 최소한의 상태만 메모리에 유지

3. 네트워크 효율성

- **멀티캐스트**: 효율적인 서비스 검색
- **캐시 제어**: 75초 캐시 타임으로 네트워크 부하 감소
- **조건부 응답**: 관련 없는 검색 요청 필터링

한계 및 개선 방안

1. 현재 한계

- **미디어 타입**: 비디오만 지원, 오디오/이미지 미지원
- **메타데이터**: 기본적인 정보만 제공 (썸네일, 자막 등 미지원)
- **트랜스코딩**: 실시간 트랜스코딩 미지원
- **고급 기능**: 북마크, 재생 위치 기억 등 미지원

2. 개선 가능 영역

- **다중 미디어 타입**: 오디오, 이미지 지원 추가
- **풍부한 메타데이터**: 썸네일, 자막, 챕터 정보 제공
- **트랜스코딩**: FFmpeg 통합으로 실시간 변환
- **검색 기능**: 제목, 장르 등으로 검색 지원
- **재생 목록**: 플레이리스트 및 큐 관리

3. 표준 준수 개선

- **UPnP AV 2.0**: 최신 표준 지원
- **DLNA 인증**: 공식 DLNA 인증 획득

- **더 많은 클라이언트**: 다양한 DLNA 클라이언트 테스트 및 최적화

결론

rqbit의 UPnP 미디어 서버 구현은 DLNA 표준을 준수하는 기능적인 미디어 서버를 제공합니다. SSDP를 통한 자동 검색, HTTP 기반 콘텐츠 제공, SOAP 제어 인터페이스를 모두 구현하여 네트워크 상의 다양한 미디어 플레이어에서 토렌트 콘텐츠를 직접 스트리밍할 수 있게 합니다.

현재는 비디오 콘텐츠에 집중되어 있지만, 모듈식 설계를 통해 향후 오디오, 이미지 지원 및 고급 기능 추가가 용이한 구조를 갖추고 있습니다. 특히 Samsung TV, VLC 등 주요 클라이언트와의 호환성을 고려한 실용적인 구현이 돋보입니다.